

EMT admin console: BFF auth plan

Reusing the personal-blog BFF auth implementation for the EMT sys-admin console. Analysis only. No BFF code has been changed.

Scope

The BFF's job is narrow: log the user in through EMT identity, keep the access token in an HttpOnly cookie, and attach it as a bearer token on calls to the .NET APIs. The .NET services do the real authorization. Reference files from personal-blog-site: `utils/httpClientUtil.ts`, `utils/authUtil.ts`, `utils/serverApi.ts`, `app/api/auth/me/route.ts`, and the login, logout and refresh routes.

1. Reuse as-is

These carry over with no design change.

Blog	EMT location	Notes
<code>httpClientUtil.ts</code> (<code>createHttpClient</code>)	<code>lib/data/client.ts</code>	Axios with a base URL, a timeout, and the self-signed-certificate agent outside production.
<code>serverApi.ts</code> <code>getAuthHeaders()</code>	<code>lib/data/client.ts</code>	Reads <code>access_token</code> from <code>cookies()</code> and turns it into <code>Authorization: Bearer</code> .
Login route: forward identity's Set-Cookie	Login Server Action	Tokens stay HttpOnly; browser code never sees them.
Logout route: delete <code>access_token</code> and <code>refresh_token</code>	Logout Server Action	Still needed, because EMT identity's logout doesn't clear cookies.
<code>normalizeUser()</code>	<code>lib/data/identity/auth.ts</code>	Keeps <code>SecurityStamp</code> and the rest of <code>IdentityUserDTO</code> out of the UI.
Only one refresh request at a time	Wherever refresh lives	More important in EMT, because its refresh tokens are single-use.

2. Tweak for EMT

Small changes to fit EMT's setup.

Blog	Tweak for EMT
One <code>BACKEND_URL</code>	Two backends: <code>IDENTITY_API_URL</code> and <code>SYSTEM_API_URL</code> , passed into <code>createHttpClient(baseUrl)</code> .
<code>getInitialUser()</code> in the root layout	Call it in <code>(dashboard)/layout.tsx</code> so the login page doesn't load the user, and redirect to <code>/login</code> when it returns null. It replaces the placeholder <code>getCurrentUser()</code> . See section 5 for the <code>cache()</code> optimization.
<code>normalizeUser</code> defaults the role to "user"	Use the real role from <code>/me</code> , since EMT only allows sys-admin.
<code>AuthProvider</code> client context	Probably not needed: the user is passed to the sidebar as a prop. Add it later only if deeper browser components need the user.

Blog	Tweak for EMT
<code>app/api/*</code> route handlers	Server Actions, as AGENTS.md asks for. Same logic, no extra HTTP hop.
Login sends <code>userName</code>	The EMT form says Email. Align the form label or identity's login field.

3. Improve

Gaps in the blog that matter more in EMT.

Gap	Improvement
Refresh only happens through the browser's axios interceptor	EMT loads most data in Server Components, which skip the interceptor. Add refresh to <code>proxy.ts</code> : if there's no access token but there is a refresh token, refresh before rendering. Without this, sessions effectively end after 30 minutes.
No route protection	<code>proxy.ts</code> sends users without a session to <code>/login</code> . The role check can stay with the .NET services.
<code>serverApi</code> turns errors into <code>[]</code> or null	Throw and show an error page instead. Otherwise a backend outage looks like "no tenants".
Identity has no <code>/me</code> endpoint	Add <code>GET /api/auth/me</code> to EMT identity, mirroring the blog backend's version (user id from the <code>NameIdentifier</code> claim) but returning id, <code>userName</code> , email and role only.

4. Server-side refresh in proxy.ts

How sessions stay alive without the browser interceptor.

Server Components can't set cookies, so the dashboard layout can't refresh a token itself. `proxy.ts` runs before every page render and Server Action, and it can set cookies, so the refresh lives there.

Request arrives with	<code>proxy.ts</code> does
Valid <code>access_token</code> , more than 1 minute left	Nothing. The request continues.
<code>access_token</code> missing, expired, or under 1 minute left, and a <code>refresh_token</code>	Calls identity <code>/api/auth/refresh</code> with the refresh cookie, copies the new <code>access_token</code> and <code>refresh_token</code> onto the response, and continues.
No usable <code>access_token</code> and no <code>refresh_token</code> , or refresh fails	Redirects to <code>/login</code> .
Valid session on <code>/login</code>	Redirects to <code>/</code> .

Reading expiry: `proxy.ts` reads the token's exp claim by base64-decoding the JWT body. It doesn't verify the signature, and doesn't need `Jwt:Key`: the value only decides *when* to refresh, never *whether* to *trust* the token. Identity's `/me` stays the real check; if it returns 401, the layout redirects to `/login`.

Why an expired token can still arrive: identity sets the cookie to expire with the token (30 minutes), so usually the cookie is simply gone. Clock differences, requests at the boundary, or a token identity rejects early (user deleted, key changed) can still send one. Refreshing a minute early also stops a token expiring partway through rendering.

Watch out: EMT refresh tokens are single-use. If two requests refresh at the same moment, the second one fails because the first already used the token. Only refreshing on page navigations, not prefetches, avoids most of this.

5. Optimization: cache() on getInitialUser

Optional. An addition on top of the blog, not a fix.

The blog calls `getInitialUser()` once, in the root layout. In EMT both the dashboard layout (sidebar) and the Overview page (welcome line, account card) need the user, and a Next layout can't pass data to its page as props. Without caching, every Overview load makes two `/me` calls.

```
export const getInitialUser = cache(async (): Promise<User | null> => { ... });
```

Aspect	Detail
Benefit	One <code>/me</code> call per request, however many components ask for the user.
Cost	One wrapper from React itself. No new dependency.
Safety	The result lives only for that one server request, so one user's data is never served to another.
If skipped	Works the same; Overview just makes one extra <code>/me</code> call.

Summary

About 70% is reused as-is. Most of the tweaks are config and file locations. The one improvement that needs real design is the server-side refresh in `proxy.ts`. The identity `/me` endpoint is a prerequisite in the other repo.